Principles of Software Architecture

MIU TechCircle

Sarah Wael,
Hana Mohamed,
Selvia Ibrahim,
Mostafa Yasser,
Ismail Ahmed

Brief Overview

MIU Circle Tech is a mobile platform designed to connect university students with internships, projects, and collaboration opportunities. The main objective of this project is to apply software architecture concepts, modeling languages (UML and ArchiMate), and architectural patterns to design a scalable and secure real-world system.

The platform enables students to create verified academic profiles showcasing their skills, background, and career interests. Through these profiles, students can discover opportunities, form teams, communicate with peers, and increase their visibility to opportunity providers.

Brief Overview(cont.)

The project focuses on analyzing functional and non-functional requirements, selecting appropriate architectural patterns, and modeling the system using multiple architectural views. Key modules include user profile management, opportunity feed, team formation, chat and communication, and academic verification.

This project demonstrates the ability to systematically design a maintainable, secure, and scalable mobile platform using established software architecture principles.

Project Objectives

- To design and develop MIU TechCircle as a centralized platform that connects students, graduates, and mentors.
- To support real-time interaction through posts, notifications, and activity updates.
- To apply software architecture patterns and modeling languages (UML, ArchiMate) in system design.
- To use a scalable and flexible software architecture that enables future system growth.
- To enhance collaboration, engagement, and information sharing within the MIU community.
- To ensure the system is maintainable, reliable, and easy to extend.

Functional Requirements

FR-1: The system shall allow users to register using an academic email address and securely log in to the platform.

FR-2: The system shall verify users' academic email addresses to confirm their student or academic status.

FR-3: The system shall allow users to reset their passwords using a secure email-based recovery process.

FR-4: The system shall allow users to create, view, and update personal profiles including name, major, academic year, university, skills, and interests.

FR-5: The system shall display a centralized Feed where internships, projects, events, and academic opportunities can be posted and browsed.

FR-6: The system shall allow users to apply for internships or submit requests to join opportunities listed in the feed.

Functional Requirements (Cont.)

FR-7: The system shall provide a Team Finder feature that allows users to search for other users based on skills, interests, or academic background.
FR-8: The system shall allow users to send, accept, or decline team invitations for competitions or collaborative activities.
FR-9: The system shall provide a unified messaging interface that allows connected users to exchange text messages and share files or documents.
FR-10: The system shall notify users of important activities including comments, friend requests, teammate or competition acceptance, reminders, and announcements.
FR-11: The system shall allow users to save posts for future reference.
FR-12: The system shall allow users to add announcements to a personal calendar.
FR-13: The system shall allow users to create and view short-lived Stories that are visible to other users for a limited period of time.

Non Functional Requirements

Non-Functional Requirement	Corresponding Quality Attribute
NFR-1: The system must load the first batch of feed posts in less than 2 seconds under normal network conditions.	Performance (Response Time)
NFR-2: User interactions such as like, comment, and repost must be reflected in the UI within 300 milliseconds.	Performance (Responsiveness)
NFR-3: The system must allow smooth continuous feed browsing without noticeable UI freezes during scrolling.	Performance (Responsiveness)
NFR-4: The backend must support growth in the number of users and posts without requiring changes to existing application logic.	Scalability
NFR-5: The system architecture must allow new content categories to be added with minimal impact on existing components.	Modifiability
NFR-6: The notification system must support thousands of concurrent subscribers per post without message loss.	Scalability (Notifications)

Non Functional Requirements (Cont.)

Non-Functional Requirement	Corresponding Quality Attribute
NFR-7: The application must continue functioning without crashing when network connectivity is temporarily lost.	Reliability
NFR-8: Unauthorized users must be prevented from accessing protected resources with 100% enforcement of role-based access rules.	Security (Access Control)
NFR-9: Messages exchanged between users must be accessible only to the authorized participants.	Security (Confidentiality)
NFR-10: All user input must be validated before submission to prevent incomplete or incorrect data entry.	Data Integrity
NFR-11: The system must allow users to view previously loaded feed data while offline.	Availability (Offline Access)
NFR-12: Like, comment, and repost counts must remain synchronized across all user views.	Data Integrity / Consistency

Non Functional Requirements (Cont.)

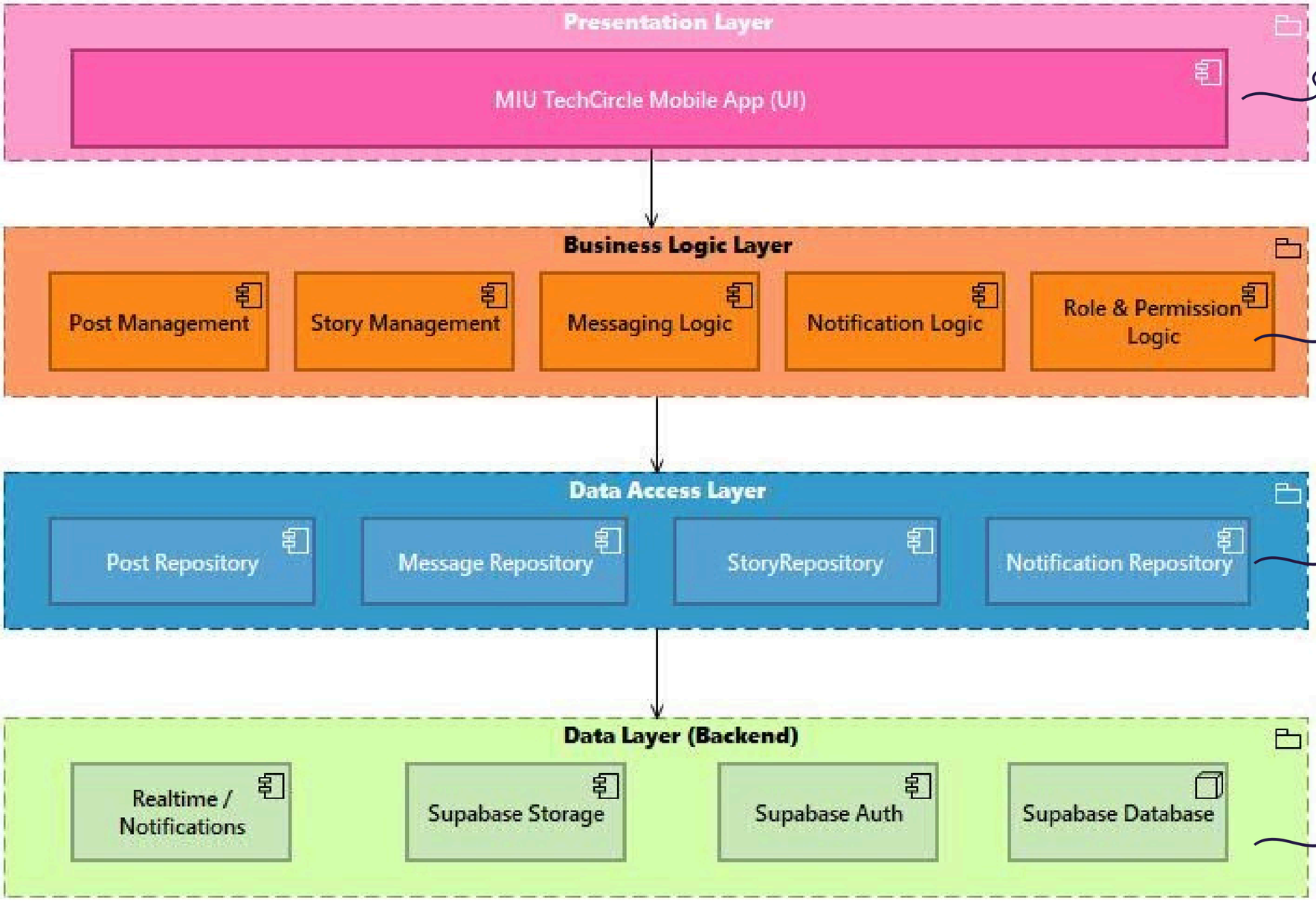
Non-Functional Requirement	Corresponding Quality Attribute
NFR-13: Only users with a verified university email domain may access core platform features.	Security (Authentication)
NFR-14: A first-time user must be able to complete signup and profile creation in $\leqslant$ 5 minutes without external assistance.	Usability (Onboarding Time)
NFR-15: Search and filtering operations must return results in $\leqslant$ 2 seconds.	Performance (Search)

Proposed Architectures

1-Abstract/Fundamental Pattern
Layered Architecture

What Is Layered Architecture?

Layered architecture is a software design approach where the system is divided into layers, each with a specific responsibility. which makes the system easier to maintain and modify. This separation improves scalability, testing, and code organization.



This layer is responsible only for displaying information and capturing user actions

This layer processes user requests received from the presentation layer and handles system behavior as it contains the core application logic

It acts as an abstraction between the business logic and the backend. It includes repositories which are responsible for retrieving and storing data.

It represents the backend infrastructure as it provides the technical foundation on which the application operates.

Why we choose it in our project ?

Supports complex features such as posts, messaging, notifications, and role-based access control.

Improves maintainability and modifiability, as changes in one layer do not affect others.

Enhances security by enforcing validation and access control in the business logic layer.

More suitable than other abstract patterns because it provides clear internal structure, not just high-level communication.

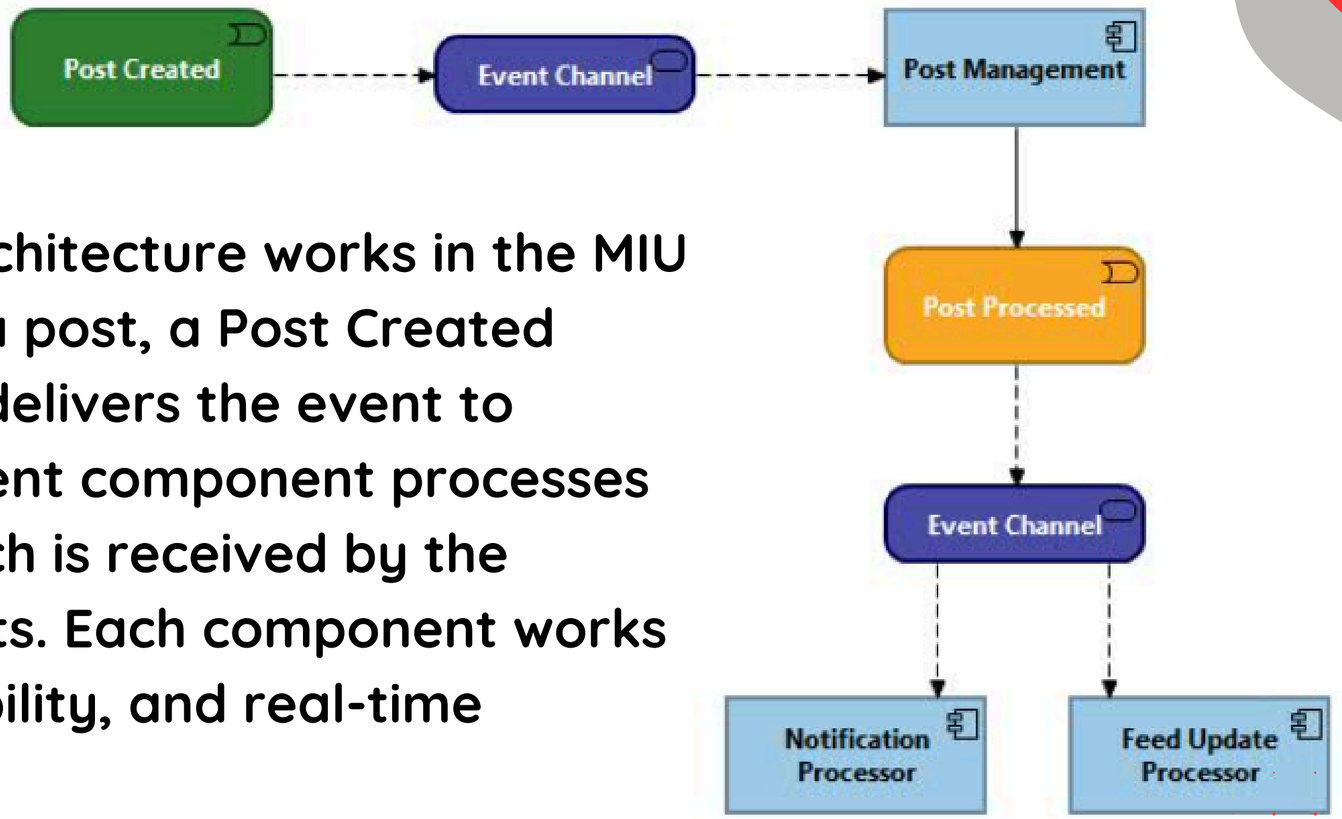
2- Specialized Pattern

Event-Driven Architecture Pattern

What Is Event-Driven Architecture Pattern?

Event-Driven Architecture is a design pattern where system parts communicate using events instead of calling each other directly.
When something happens, an event is created, and other parts of the system respond to it automatically.

Event Driven Diagram



This diagram shows how Event-Driven Architecture works in the MIU TechCircle system. When a user creates a post, a Post Created event is sent to an event channel, which delivers the event to system components. The Post Management component processes it and sends a Post Processed event, which is received by the Notification and Feed Update components. Each component works independently, enabling scalability, flexibility, and real-time updates.

Final ArchiteActure Choosen

Layered Architecture

Performance & Responsiveness

- Separates UI from heavy backend processing to ensure fast response times and smooth scrolling (NFR-1, 2, 3, 15).
- Enables efficient caching and pagination without UI lag.

Scalability & Modifiability

- Independent scaling of backend services to handle thousands of concurrent users (NFR-4, 6).
- Isolates business rules, allowing for easy feature updates with minimal refactoring (NFR-5).

Final ArchiteActure Choosen

Layered Architecture (Continued)

Security & Data Integrity

- Centralized authentication and authorization within the Logic Layer (NFR-8, 9, 13).
- Enforces strict validation rules to ensure data consistency across the platform (NFR-10, 12).

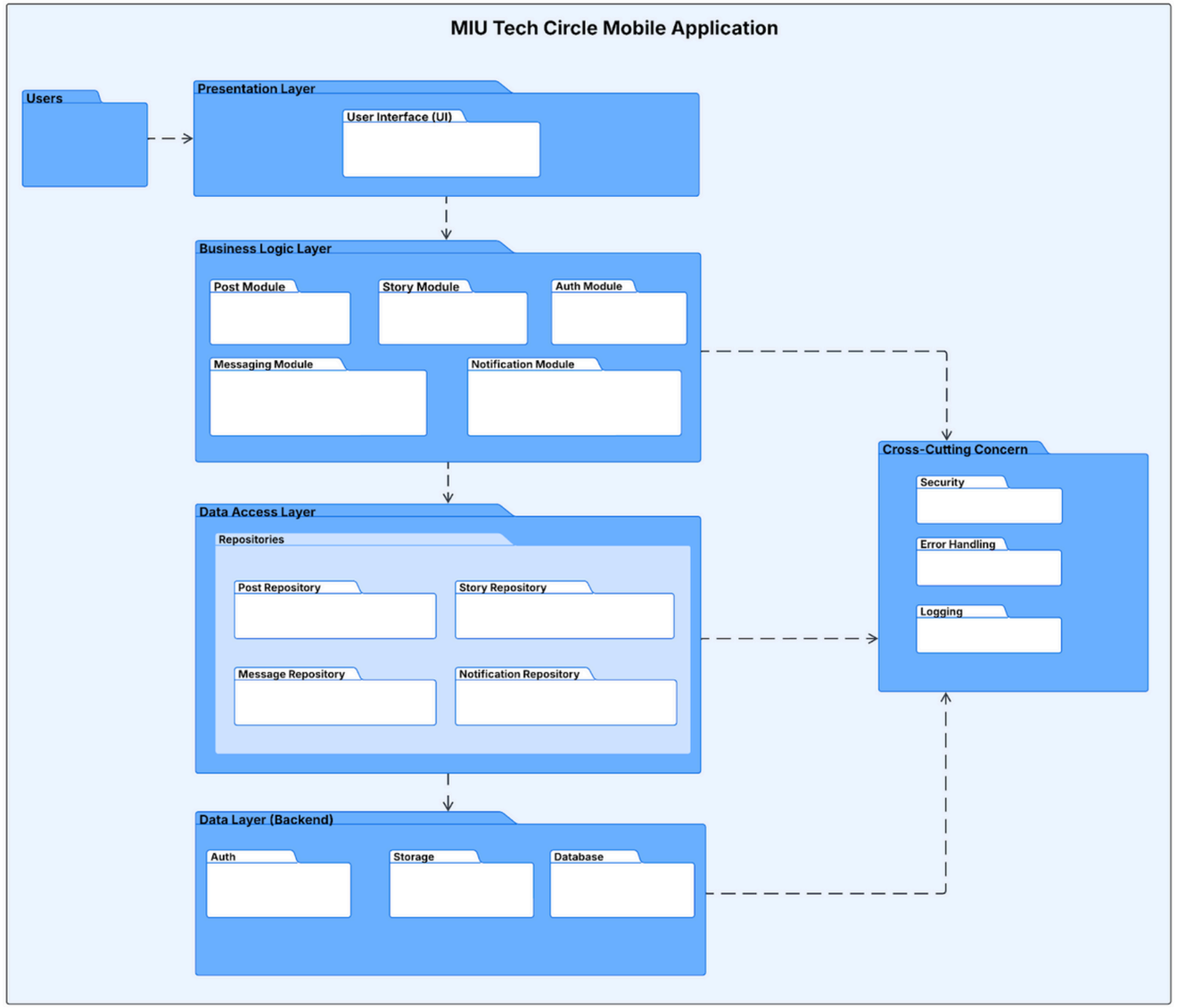
Reliability & Availability

- Decoupled layers allow for graceful failure handling and offline access via cached data (NFR-7, 11).

Architectural Views

Logical View

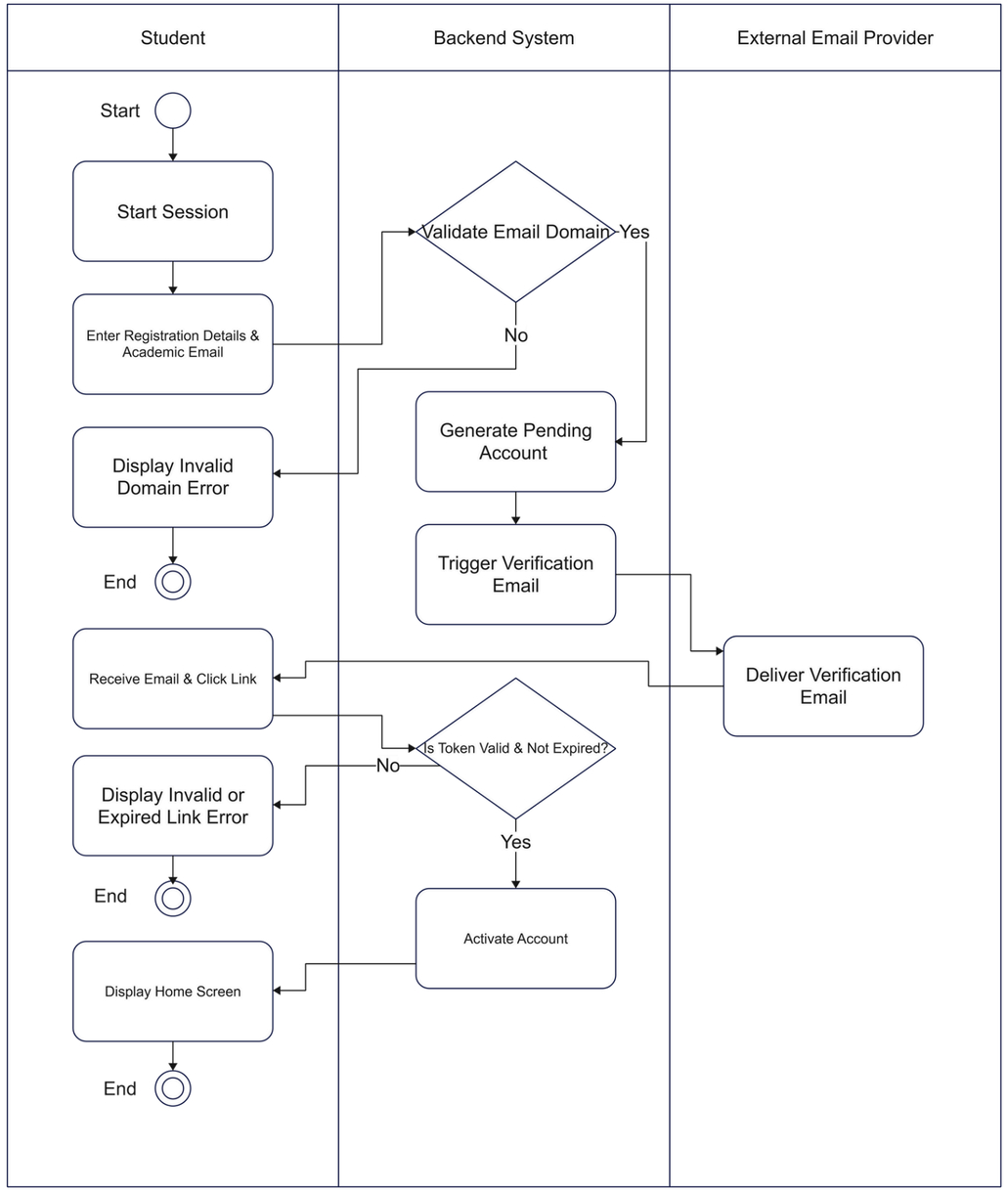
It describes what the system is made of logically and how its main parts are organized and related, without focusing on hardware, deployment, or technology details.



Process View

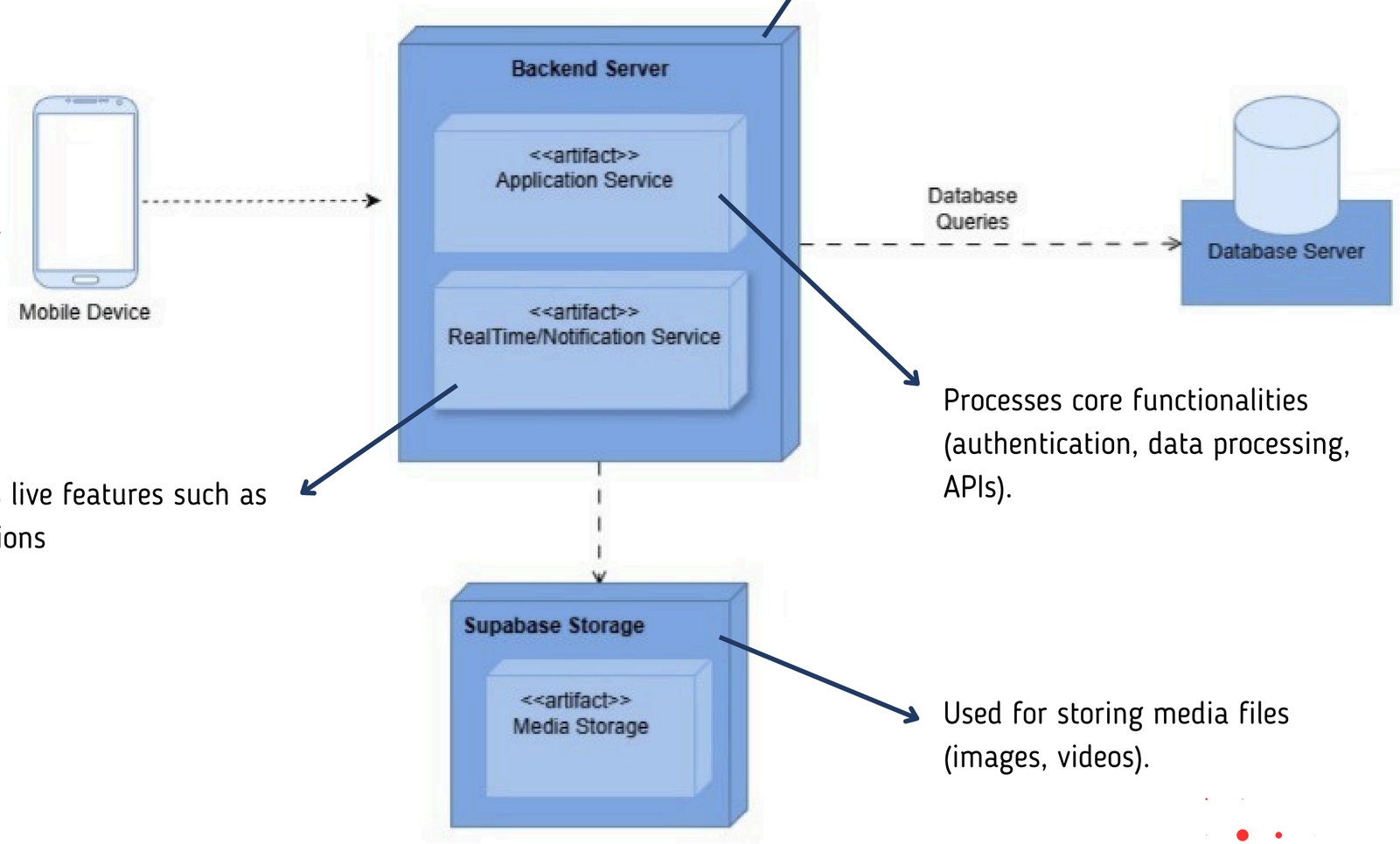
Workflow Highlights

- Domain Validation: Restricts access to verified university emails for community security.
- Pending State: Prevents unauthorized access until verification is complete.
- Integration: Uses external provider to decouple email delivery from core logic.
- Token Security: Employs time-sensitive links to prevent link reuse or expiration errors.



Deployment View

Central component that handles all business logic that acts as a bridge between the mobile app, database, and storage.



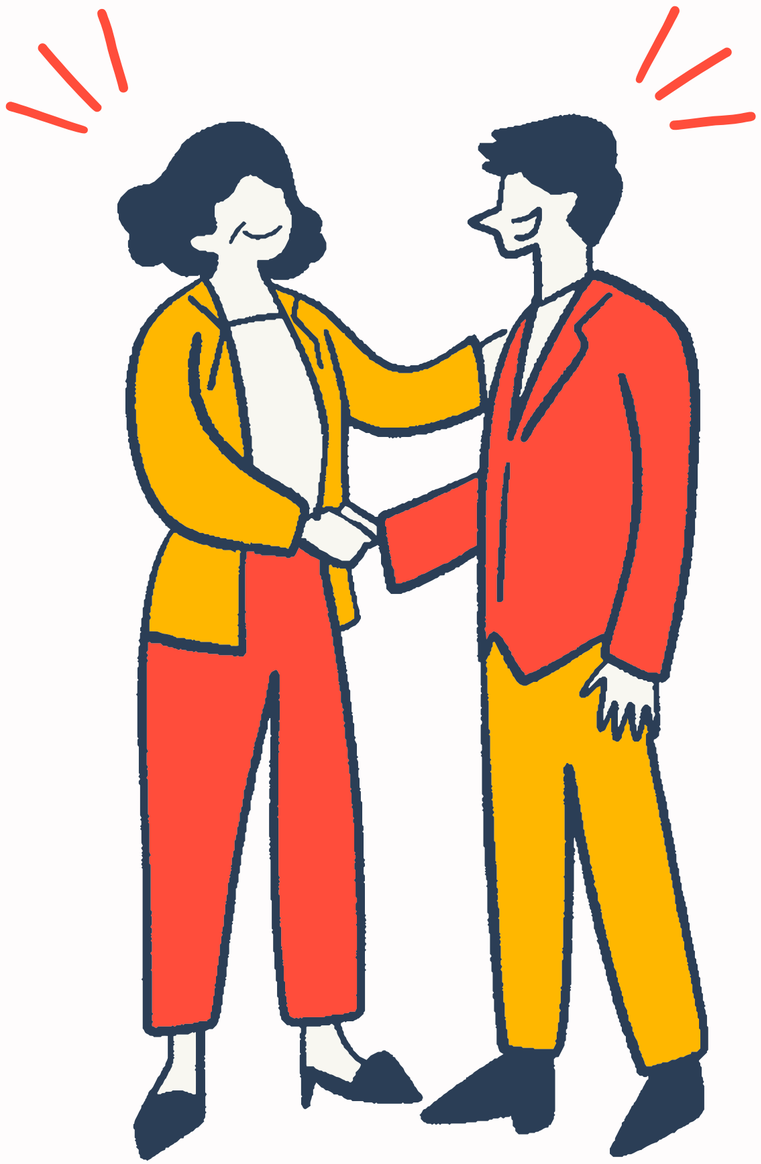
Sends user requests (login, posts, actions) to the backend.

Manages live features such as notifications

Processes core functionalities (authentication, data processing, APIs).

Stores structured data (users, posts, interactions).

Used for storing media files (images, videos).



Conclusion

-Designed the architecture of MIU Circle Tech, a student collaboration platform.
-Applied software architecture principles using UML and ArchiMate.
-Analyzed functional and non-functional requirements.
-Selected a Layered Architecture for scalability and maintainability.
-Supported real-time interactions through event-driven concepts.
-Modeled the system using multiple architectural views.
-Ensured security and academic verification.
-Proposed an architecture ready for future expansion.

Thank you